



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

Department of Computer Science and Engineering

2025-2026

Odd Semester

DATA STRUCTURES AND ALGORITHMS-3

(25CS2103E)

Optimized Project Task Scheduling System

M. Sai Bhavitha Reddy

2520030551

I. Aswika

2520030221

V. Hamsini

2520030556

COURSE INSTRUCTOR

Dr. J Sirisha Devi

Professor

Department of Computer Science and Engineering

OPTIMIZED PROJECT TASK SCHEDULING SYSTEM

Team 19 | Section 5 | Guide: D. Swathi

1. Abstract

The Project Scheduling System is designed to efficiently manage and optimize the execution of tasks in a project while satisfying the dependencies between activities. Each task may depend on one or more other tasks, which must be completed before it can begin. The system uses data structures and optimization techniques to determine a valid and efficient execution order for all tasks. It generates an optimized project schedule with the objective of minimizing the overall completion time. The system also identifies critical tasks and the critical path, which are important for completing the project on time. The execution order of tasks is displayed clearly to help users understand the project workflow. This project demonstrates the practical application of concepts such as graphs, topological sorting, dependency management, and critical path analysis in project management. The system can help improve task planning, scheduling efficiency, and overall project management.

2. Introduction

Every real project, whether it is building software, constructing a bridge or organising an event, is made of many tasks, and some of them cannot start until others have finished. Testing cannot begin before development is complete, and development cannot begin before the database has been designed and set up. A project manager must answer three questions: **in what order should the tasks be done, what is the shortest time in which the whole project can be finished, and which tasks can never be delayed without delaying the project?** Answering these by hand becomes slow and error-prone as soon as the number of tasks grows.

A **Directed Acyclic Graph (DAG)** models this naturally. Every task is a node and every dependency is a directed edge from the task that must finish first to the task that waits for it. "Acyclic" means that no task can directly or indirectly depend on itself; a circular dependency makes a schedule impossible and must be detected. On the DAG, **Topological Sort** gives a valid execution order and the **Critical Path Method (CPM)** gives the minimum project duration. CPM computes the following values for every task v :

- **ES (earliest start)** = the maximum EF of all predecessors (0 if there is none).
- **EF (earliest finish)** = ES + duration.
- **LF (latest finish)** = the minimum LS of all successors (the project end time if there is none).
- **LS (latest start)** = LF - duration.
- **Slack** = LS - ES. A task with slack 0 is **critical**: any delay in it delays the whole project. A task with positive slack has room to slip by that many days.

Real projects also need more than a schedule: finding tasks by name, spotting duplicate tasks, deciding who does what, planning with a limited number of workers and handling uncertain durations. This project therefore builds a menu-driven Java application in which CPM is the

core and the remaining needs are solved with the algorithm families studied in DSA-3 (string algorithms, advanced dynamic programming, network flow, approximation, randomised and parallel algorithms).

3. Problem Statement and Objectives

Problem statement. Given a set of tasks with durations and dependencies, find a valid order of execution, the minimum completion time and the critical tasks, and support staffing decisions, without manual calculation.

Objectives

- Model tasks and dependencies as a DAG and reject circular dependencies using DFS.
- Produce a valid execution order using Topological Sort (Kahn's algorithm).
- Compute ES, EF, LS, LF, slack, the critical path and the minimum project duration (CPM).
- Search task names (KMP) and flag near-duplicate task names (suffix array + LCP array).
- Plan work with dynamic programming, assign workers by max-flow and approximate m-worker schedules.
- Estimate schedule risk with Monte Carlo simulation and use parallel prefix sum with work-span analysis.

4. Case Study: A Five-Task Software Project

To explain the system we use a small software project of five tasks. Duration is in days. This is the same sample project built into the program (menu option 2), so every number below can be reproduced from the code.

ID	Task	Duration (days)	Depends on
A	Requirements	2	-
B	Database Design	3	A
C	Database Setup	2	A
D	Development	4	B, C
E	Testing	2	D

Step 1: Build the DAG and check for cycles. The edges are $A \rightarrow B$, $A \rightarrow C$, $B \rightarrow D$, $C \rightarrow D$ and $D \rightarrow E$. The in-degrees are $A = 0$, $B = 1$, $C = 1$, $D = 2$ and $E = 1$. A DFS with three colours (unvisited, on the stack, finished) finds no back edge, so the graph is a valid DAG. Had the input made A depend on E, the program would stop with the message "circular dependency detected: A -> B -> D -> E -> A".

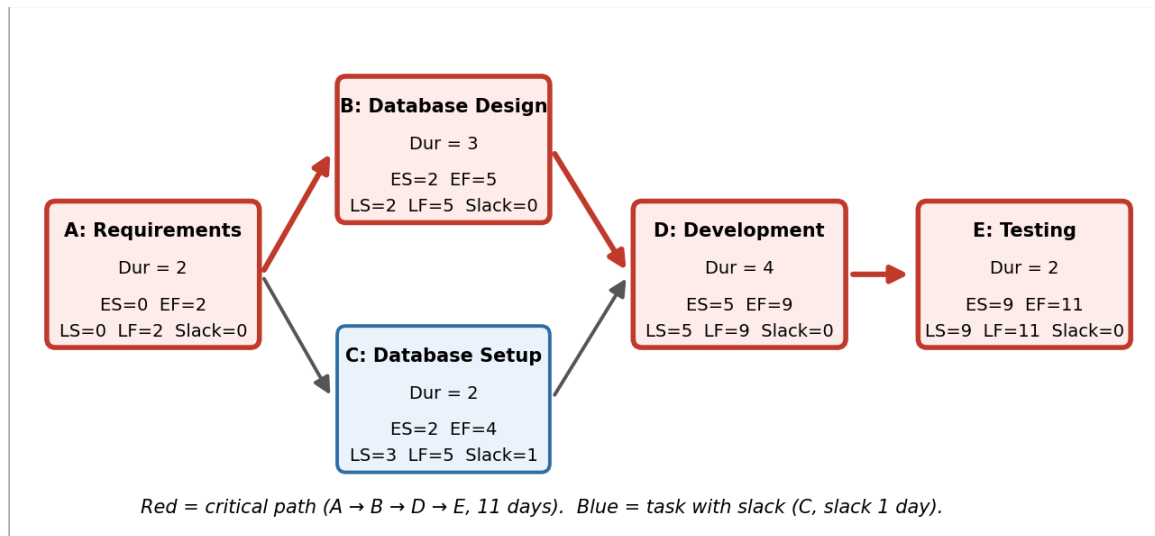


Figure 1: DAG of the sample project with CPM values (red = critical path)

Step 2: Topological sort. Kahn's algorithm starts with the tasks of in-degree 0, which is only A. After A is removed, B and C become ready; a min-heap picks the smallest ID first. D becomes ready only after both B and C are done, and then E follows. The execution order is **A → B → C → D → E**.

Step 3: Forward pass. In topological order, ES is the largest EF among the predecessors. A: ES 0, EF 2. B: ES 2, EF 5. C: ES 2, EF 4. D waits for both B (EF 5) and C (EF 4), so ES = 5 and EF = 9. E: ES 9, EF 11. The earliest the project can finish is **day 11**.

Step 4: Backward pass. In reverse order, LF is the smallest LS among the successors, and the last task gets LF = 11. E: LS 9. D: LF 9, LS 5. C: LF 5, LS 3. B: LF 5, LS 2. A: LF = min(LS of B, LS of C) = 2, LS 0.

Step 5: Slack and critical path. Slack = LS - ES gives A = 0, B = 0, C = 1, D = 0, E = 0. The result is shown below.

ID	Task	Dur	ES	EF	LS	LF	Slack	Critical
A	Requirements	2	0	2	0	2	0	YES
B	Database Design	3	2	5	2	5	0	YES
C	Database Setup	2	2	4	3	5	1	no
D	Development	4	5	9	5	9	0	YES
E	Testing	2	9	11	9	11	0	YES

The **critical path** is A → B → D → E with length 2 + 3 + 4 + 2 = **11 days**. The other route, A → C → D → E, takes only 10 days, which is exactly the one day of slack on task C.

What the manager learns. Any delay in A, B, D or E delays the whole project day for day, so these tasks need the most attention. Task C can slip by one day without any effect. To finish earlier, only critical tasks should be shortened: reducing Development from 4 to 3 days brings the project to 10 days, whereas shortening Database Setup changes nothing.

Further analysis on the same project. The same DAG is reused by the other menu options. With only one worker the work takes 13 days (the sum of all durations); with two workers the

project still takes 11 days, which equals the lower bound; the names "Database Design" and "Database Setup" share the text "database ", so the system warns that they may be duplicates; and a Monte Carlo run with random delays shows that the realistic expected duration is about 13.75 days rather than 11. The measured outputs are given in the Results section.

5. System Overview

The program follows the pipeline shown in Figure 2. After the schedule is printed, a menu lets the user run the analysis modules on the same DAG. Tasks are stored in a hash map (ID → task) for constant-time lookup and the dependencies in an adjacency list; the in-degree of every node is kept for Kahn's algorithm.

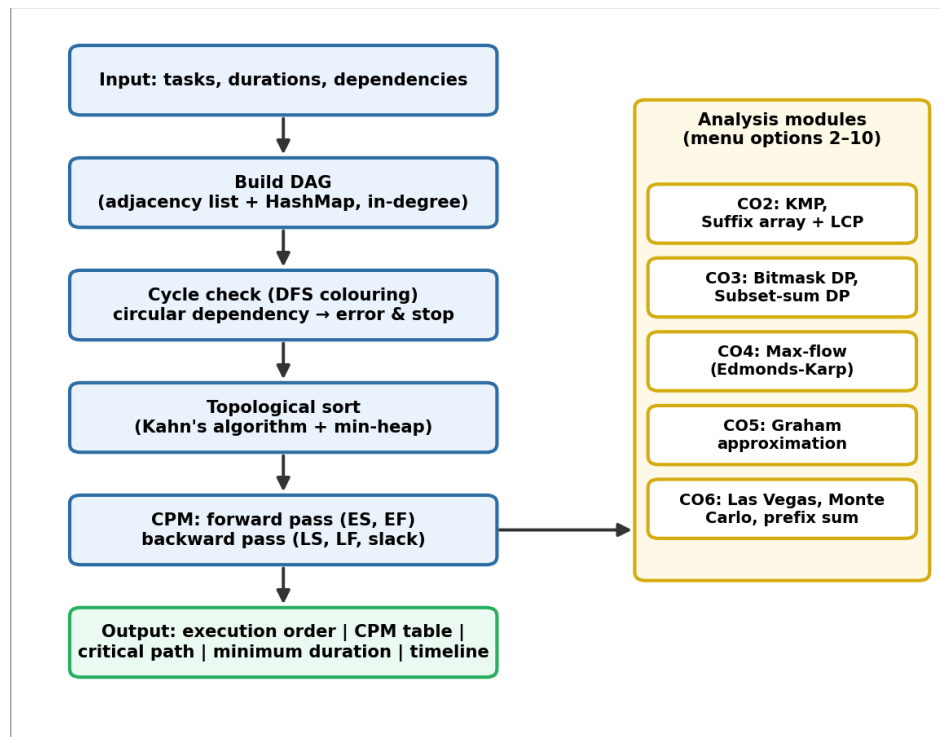


Figure 2: Flow chart of the Project Scheduling System

Menu option	Function	Technique
1	Show schedule again	Topological order + CPM table
2	Search tasks by keyword	KMP string matching
3	Find similar task names	Suffix array + LCP array
4	Best order for a single worker	Bitmask DP ($n \leq 20$) / greedy
5	Split work between 2 teams	Subset-sum DP
6	Algorithm selection report	Problem-class analysis
7	Assign tasks to workers	Max-flow (Edmonds-Karp), min-cut
8	Schedule on m workers	Graham list scheduling (approximation)
9	Randomised + parallel analysis	Las Vegas, Monte Carlo, prefix sum
10	Run everything	Full demo report

6. Literature Survey

- Kelley and Walker (1959) introduced the Critical Path Method; Malcolm et al. (1959) introduced PERT for the Polaris programme.
- Kahn (1962) gave the $O(V+E)$ topological sorting algorithm for a DAG.
- Knuth, Morris and Pratt (1977) gave linear-time string matching (KMP).
- Ford and Fulkerson (1956) and Edmonds and Karp (1972) developed maximum flow and the max-flow min-cut theorem.
- Graham (1966) showed that list scheduling is within a factor $(2 - 1/m)$ of the optimum on m machines.
- Ullman (1975) showed that scheduling with precedence constraints is NP-complete in general.
- Cormen, Leiserson, Rivest and Stein, "Introduction to Algorithms" is the reference text for the algorithms used.

7. Course Outcome (CO1-CO6) Coverage

CO	Topic	Where it is used in the project
CO1	Problem class and strategy selection	KMP chosen over naive search; bitmask DP for $n \leq 20$ and greedy above; max-flow for assignment; option 6 prints the problem-class table
CO2	String algorithms	KMP keyword search over task names; suffix array + LCP (Kasai) to find similar task names
CO3	Advanced dynamic programming	CPM as DP over the DAG; bitmask DP for the best single-worker order and number of valid orders; subset-sum DP for the two-team split
CO4	Network flow	Edmonds-Karp max-flow for task-worker assignment; min-cut check
CO5	NP-hardness and approximation	Graham list scheduling on m workers with a $(2 - 1/m)$ guarantee and a lower bound
CO6	Randomised and parallel algorithms	Las Vegas quick-select; Monte Carlo simulation; parallel reduce and prefix sum with work-span analysis

8. Hardware and Software Requirements

Hardware

- Any laptop or PC with a dual-core CPU and 4 GB RAM; less than 10 MB of disk space; no GPU or network needed.

Software

- Java JDK 11 or newer (tested on JDK 21); Windows, Linux or macOS.
- VS Code, IntelliJ or Eclipse (or a terminal). Run with: `javac ProjectScheduler.java`, then `java ProjectScheduler`.

ALGORITHM / PSEUDO CODE

The algorithms are given in the order in which the program uses them. Each pseudo code is followed by its running time. V is the number of tasks and E the number of dependencies.

1. Main driver

```
PROGRAM ProjectScheduler
1. READ the project (own input or the built-in sample A-E)
2. FOR each task t AND each predecessor p IN t.preds
   IF p does not exist OR p = t THEN report error and STOP
   ADD edge p -> t (adj[p].add(t), indegree[t]++)
3. cycle <- FIND_CYCLE(G)
   IF cycle exists THEN print "circular dependency" and STOP
4. order <- TOPOLOGICAL_SORT(G)
5. total <- RUN_CPM(G, order)
6. PRINT order, CPM table, critical path(s), total, timeline
7. REPEAT: show the menu and run the chosen analysis module UNTIL user selects Exit
```

2. Cycle detection (DFS colouring), $O(V + E)$

```
FUNCTION FIND_CYCLE(G)
  color[v] <- WHITE for every task v ; stack <- empty
  FOR each task u in G
    IF color[u] = WHITE THEN
      c <- DFS(u)
      IF c is not NULL THEN RETURN c
  RETURN NULL

FUNCTION DFS(u)
  color[u] <- GREY ; PUSH u on stack
  FOR each v in adj[u]
    IF color[v] = GREY THEN // back edge = cycle
      cycle <- stack items from top down to v, reversed, then v
      RETURN cycle
    IF color[v] = WHITE THEN
      c <- DFS(v) ; IF c is not NULL THEN RETURN c
  POP stack ; color[u] <- BLACK
  RETURN NULL
```

3. Topological sort (Kahn's algorithm + min-heap), $O(V \log V + E)$

```
FUNCTION TOPOLOGICAL_SORT(G)
  deg <- copy of indegree
  ready <- min-heap of IDs ; order <- empty list
  FOR each task v with deg[v] = 0 DO ready.add(v)
  WHILE ready is not empty
    u <- ready.removeMin()
    order.append(u)
    FOR each v in adj[u]
      deg[v] <- deg[v] - 1
      IF deg[v] = 0 THEN ready.add(v)
  RETURN order
```

4. Critical Path Method (DP over the topological order), $O(V + E)$

```
FUNCTION RUN_CPM(G, order)
  end <- 0
  // forward pass: sub-problems are solved before they are needed
  FOR each id in order (left to right)
    t.ES <- max( EF[p] for p in t.preds ) (0 if no predecessor)
    t.EF <- t.ES + t.duration
    end <- max(end, t.EF)
  // backward pass
  FOR each id in order (right to left)
    t.LF <- min( LS[s] for s in adj[t] ) (end if no successor)
```

```

    t.LS    <- t.LF - t.duration
    t.slack <- t.LS - t.ES
    RETURN end                                     // minimum project duration

FUNCTION COLLECT_PATHS(u, current, out)           // critical path extraction
  current.add(u) ; extended <- false
  FOR each v in adj[u]
    IF v.slack = 0 AND v.ES = u.EF THEN COLLECT_PATHS(v, current, out) ; extended <- true
  IF NOT extended THEN out.add(current as "A -> B -> ...")
  current.removeLast()
// called for every critical task with ES = 0

```

5. KMP keyword search over task names, $O(n + m)$

```

FUNCTION PREFIX_FUNCTION(P)                       // failure function
  pi[0] <- 0 ; k <- 0
  FOR i <- 1 TO |P|-1
    WHILE k > 0 AND P[i] != P[k] DO k <- pi[k-1]
    IF P[i] = P[k] THEN k <- k + 1
    pi[i] <- k
  RETURN pi

FUNCTION KMP_SEARCH(T, P)
  pi <- PREFIX_FUNCTION(P) ; k <- 0 ; result <- empty
  FOR i <- 0 TO |T|-1
    WHILE k > 0 AND T[i] != P[k] DO k <- pi[k-1]
    IF T[i] = P[k] THEN k <- k + 1
    IF k = |P| THEN result.add(i-k+1) ; k <- pi[k-1]
  RETURN result
// keywordSearch lower-cases the keyword and every task name, then calls KMP_SEARCH for each task

```

6. Similar task names (suffix array + LCP array)

```

FUNCTION SIMILAR_NAMES(G, minLen)
  S <- all lower-cased task names joined, each followed by a unique separator character
  owner[i] <- index of the task that owns character i of S
  SA <- suffix array of S (suffixes sorted)
  LCP <- Kasai's algorithm: rank[SA[i]] <- i
  h <- 0
  FOR i <- 0 TO n-1
    IF rank[i] > 0 THEN
      j <- SA[rank[i]-1]
      WHILE S[i+h] = S[j+h] DO h <- h + 1
      LCP[rank[i]] <- h ; IF h > 0 THEN h <- h - 1
    ELSE h <- 0
  FOR each i, scanning j = i+1, i+2, ...
    m <- min(m, LCP[j]) // range-minimum = common prefix length
    IF m < minLen THEN BREAK
    IF owner[SA[i]] and owner[SA[j]] are different tasks THEN record the pair with length m
  PRINT every recorded pair and the shared text

```

7. Single worker: bitmask DP ($n \leq 20$) and greedy heuristic

```

FUNCTION BITMASK_PLAN(order)
  predMask[i] <- bit mask of the predecessors of task i ; sum[mask] <- total duration of tasks in mask
  dp[0] <- 0 ; ways[0] <- 1 ; dp[other] <- INFINITY
  FOR mask <- 0 TO  $2^n - 1$  (skip if dp[mask] = INFINITY)
    FOR each task i not in mask
      IF (predMask[i] AND mask) = predMask[i] THEN // task i is ready
        nm <- mask OR (1 << i)
        cost <- dp[mask] + sum[nm] // task i finishes at sum[nm]
        IF cost < dp[nm] THEN dp[nm] <- cost ; parent[nm] <- i
        ways[nm] <- ways[nm] + ways[mask] // counts topological orders
  rebuild the best order from parent[] starting at the full mask
  RETURN dp[full], ways[full] // time  $O(2^n * n)$ 

FUNCTION GREEDY_PLAN(G) // used when  $n > 20$ 
  ready <- priority queue by (duration, ID) of tasks with in-degree 0

```

```

WHILE ready is not empty
  t <- ready.removeMin() ; time <- time + t.duration ; sumFinish <- sumFinish + time
  FOR each successor v of t: reduce its in-degree ; IF it becomes 0 THEN ready.add(v)

```

8. Two-team split (subset-sum DP), $O(n \times \text{total}/2)$

```

FUNCTION TWO_TEAM_SPLIT(G)
  total <- sum of durations ; half <- total / 2
  can[0][0] <- true
  FOR i <- 1 TO n, FOR s <- 0 TO half
    can[i][s] <- can[i-1][s] OR (s >= d_i AND can[i-1][s-d_i])
  best <- largest s <= half with can[n][s] = true
  trace back: task i goes to Team 1 if can[i-1][s] is false, then s <- s - d_i
  Team 1 workload = best ; Team 2 workload = total - best

```

9. Task-to-worker assignment (max-flow, Edmonds-Karp), $O(V E^2)$

```

FUNCTION ASSIGN_WORKERS(G, workers)
  build network: source -> worker (capacity = max tasks)
                  worker -> task (capacity 1 if the worker has the skill)
                  task -> sink (capacity 1)
  flow <- EDMONDS_KARP(c, source, sink)

FUNCTION EDMONDS_KARP(c, s, t)
  total <- 0
  LOOP
    BFS from s over edges with c[u][v] > 0, storing parent[]
    IF t was not reached THEN RETURN total
    f <- minimum residual capacity along the path s ... t
    FOR each edge (u,v) on the path: c[u][v] <- c[u][v] - f ; c[v][u] <- c[v][u] + f
    total <- total + f
  // afterwards: tasks with flow = assignments; unassigned tasks = tasks still linked to the sink
  // min-cut = capacity of original edges from the residual-reachable side to the other side;
  // the program checks that max-flow = min-cut

```

10. m identical workers (Graham list scheduling, $(2 - 1/m)$ -approximation)

```

FUNCTION MULTI_WORKER_SCHEDULE(G, m, cpmTotal)
  free[1..m] <- 0 ; time <- 0 ; remaining <- all tasks
  WHILE remaining is not empty
    ready <- tasks whose predecessors have all finished by 'time'
    sort ready by duration (longest first), ties by ID
    FOR each task t in ready
      w <- first worker with free[w] <= time ; IF none THEN BREAK
      start t on w at 'time' ; finish[t] <- time + d_t ; free[w] <- finish[t]
    time <- smallest finish time greater than 'time'
  makespan <- max finish[t]
  lowerBound <- max( cpmTotal, ceil(totalWork / m) )
  PRINT makespan, lowerBound, makespan / lowerBound, guarantee 2 - 1/m

```

11. Randomised and parallel algorithms

```

FUNCTION QUICK_SELECT(a, k) // Las Vegas: always correct, random running time
  lo <- 0 ; hi <- n-1
  LOOP
    IF lo = hi THEN RETURN a[lo]
    pivot <- a[random index in lo..hi] ; partition a[lo..hi] around pivot (i, j meet)
    IF k <= j THEN hi <- j ELSE IF k >= i THEN lo <- i ELSE RETURN a[k]

FUNCTION MONTE_CARLO(G, order, trials = 10000) // Monte Carlo: random, approximate
  FOR r <- 1 TO trials
    FOR each task in topological order
      d' <- duration x (1 + 0.5 x random()) // 0% to 50% random delay
      EF <- max(EF of predecessors) + d'
    record project end time, whether end <= deadline, and the tasks on the longest path

```

```

RETURN average end time, P(end <= deadline), criticality index of each task

FUNCTION PREFIX_SUM(dur)                                // parallel scan (Hillis-Steele)
  FOR d <- 1 TO n-1 STEP d <- 2d                        // ceil(log2 n) rounds
    FOR EACH i >= d IN PARALLEL: next[i] <- scan[i] + scan[i-d]
    scan <- next
  // work O(n log n) for this scan, span O(log n); reduce uses n-1 additions with span O(log n)

```

12. Complexity summary

Module	Technique	Time complexity
Cycle check	DFS colouring	$O(V + E)$
Execution order	Kahn + min-heap	$O(V \log V + E)$
CPM	DP over topological order	$O(V + E)$
Keyword search	KMP	$O(n + m)$
Similar names	Suffix array + Kasai LCP	sorting suffixes + $O(n)$ LCP
Single worker	Bitmask DP / greedy	$O(2^n n)$ for $n \leq 20$ / $O(V \log V + E)$
Two-team split	Subset-sum DP	$O(n \times \text{total}/2)$
Worker assignment	Edmonds-Karp	$O(V E^2)$
m workers	Graham list scheduling	polynomial, $(2 - 1/m)$ -approximation
Median duration	Randomised quick-select	expected $O(n)$
Prefix sum / reduce	Parallel scan	span $O(\log n)$

CODE

File: ProjectScheduler.java (Java, single file, 685 lines)

```
import java.util.*;

/**
 * Project Scheduling System Using DAG and CPM          DSA-3 (25CS2103E)
 *
 * Core pipeline : Input -> DAG -> cycle check (DFS) -> Topological Sort (Kahn) -> CPM
 *
 * Course-outcome topics used
 * CO1 Evaluate problem class & select strategy
 *      - substring search          -> KMP chosen (linear time)
 *      - NP-hard scheduling        -> exact bitmask DP for small n, greedy heuristic for
large n
 * CO2 Linear-time string algorithms / suffix structures
 *      - KMP                      -> keyword search over task names
 *      - Suffix Array + LCP array  -> detect similar / duplicate task names
 * CO4 Max-flow (Edmonds-Karp) + min-cut check -> assign tasks to workers
 * CO5 NP-hard scheduling -> Graham (2-1/m) approximation, lower-bound ratio
 * CO6 Las Vegas quick-select, Monte Carlo simulation, parallel reduce/prefix sum (work-span)
 * CO3 Advanced DP patterns
 *      - Bitmask DP / DP on subsets -> best single-worker order + count of valid orders
 *      - DP on subsets (subset-sum) -> split workload between 2 teams
 *      - DP on a DAG (tree-DP idea)  -> CPM forward/backward passes
 */
public class ProjectScheduler {

    // =====
    // Task and DAG
    // =====
    static class Task {
        String id, name;
        int duration;
        List<String> preds = new ArrayList<>();
        int es, ef, ls, lf, slack;
        Task(String id, String name, int duration) { this.id = id; this.name = name; this.duration
= duration; }
        boolean isCritical() { return slack == 0; }
    }

    static class DAG {
        Map<String, Task> tasks = new LinkedHashMap<>(); // hashing: id -> task
        Map<String, List<String>> adj = new HashMap<>(); // adjacency list
        Map<String, Integer> indegree = new HashMap<>();

        void addTask(Task t) { tasks.put(t.id, t); adj.put(t.id, new ArrayList<>());
indegree.put(t.id, 0); }

        void addEdge(String from, String to) { adj.get(from).add(to); indegree.merge(to, 1,
Integer::sum); }

        // DFS cycle detection (0 = unvisited, 1 = on stack, 2 = finished)
        List<String> findCycle() {
            Map<String, Integer> color = new HashMap<>();
            Deque<String> stack = new ArrayDeque<>();
            for (String id : tasks.keySet())
                if (color.getOrDefault(id, 0) == 0) {
                    List<String> c = dfs(id, color, stack);
                    if (c != null) return c;
                }
            return null;
        }

        private List<String> dfs(String u, Map<String, Integer> color, Deque<String> stack) {
            color.put(u, 1); stack.push(u);
            for (String v : adj.get(u)) {
                if (color.getOrDefault(v, 0) == 1) {
                    List<String> cycle = new ArrayList<>();
                    for (String s : stack) { cycle.add(s); if (s.equals(v)) break; }
                    Collections.reverse(cycle); cycle.add(v);
                    return cycle;
                }
            }
        }
    }
}
```

```

        }
        if (color.getDefault(v, 0) == 0) { List<String> c = dfs(v, color, stack); if (c
!= null) return c; }
    }
    stack.pop(); color.put(u, 2);
    return null;
}

// Kahn's algorithm; the min-heap picks the smallest ID among ready tasks
List<String> topologicalSort() {
    Map<String, Integer> deg = new HashMap<>(indegree);
    PriorityQueue<String> ready = new PriorityQueue<>();
    for (String id : tasks.keySet()) if (deg.get(id) == 0) ready.add(id);
    List<String> order = new ArrayList<>();
    while (!ready.isEmpty()) {
        String u = ready.poll();
        order.add(u);
        for (String v : adj.get(u)) { deg.put(v, deg.get(v) - 1); if (deg.get(v) == 0)
ready.add(v); }
    }
    return order;
}

// =====
// CPM (DP over the topological order)
// =====
static int runCPM(DAG g, List<String> order) {
    int end = 0;
    for (String id : order) { // forward pass
        Task t = g.tasks.get(id);
        t.es = 0;
        for (String p : t.preds) t.es = Math.max(t.es, g.tasks.get(p).ef);
        t.ef = t.es + t.duration;
        end = Math.max(end, t.ef);
    }
    for (int i = order.size() - 1; i >= 0; i--) { // backward pass
        Task t = g.tasks.get(order.get(i));
        t.lf = end;
        for (String s : g.adj.get(t.id)) t.lf = Math.min(t.lf, g.tasks.get(s).ls);
        t.ls = t.lf - t.duration;
        t.slack = t.ls - t.es;
    }
    return end;
}

static void collectPaths(DAG g, String u, Deque<String> cur, List<String> out) {
    cur.addLast(u);
    boolean extended = false;
    for (String v : g.adj.get(u)) {
        Task tv = g.tasks.get(v);
        if (tv.isCritical() && tv.es == g.tasks.get(u).ef) { collectPaths(g, v, cur, out);
extended = true; }
    }
    if (!extended) out.add(String.join(" -> ", cur));
    cur.removeLast();
}

static void printSchedule(DAG g, List<String> order, int total) {
    System.out.println("\n=== EXECUTION ORDER (Topological Sort) ===");
    System.out.println(String.join(" -> ", order));

    System.out.println("\n=== CPM SCHEDULE ===");
    System.out.printf("%-6s %-22s %4s %4s %4s %4s %6s %s\n", "ID", "Task", "Dur", "ES",
"EF", "LS", "LF", "Slack", "Critical");
    for (String id : order) {
        Task t = g.tasks.get(id);
        System.out.printf("%-6s %-22s %4d %4d %4d %4d %6d %s\n", t.id, t.name, t.duration,
t.es, t.ef, t.ls, t.lf, t.slack,
t.isCritical() ? "YES *" : "no");
    }

    System.out.println("\n=== CRITICAL PATH(S) ===");
    List<String> paths = new ArrayList<>();
    for (String id : order) { Task t = g.tasks.get(id); if (t.isCritical() && t.es == 0)
collectPaths(g, id, new ArrayDeque<>(), paths); }
    for (String p : paths) System.out.println(p);
    System.out.println("\nMinimum project duration: " + total + " days");
}

```

```

        System.out.println("\n=== TIMELINE (# = critical, - = has slack) ===");
        for (String id : order) {
            Task t = g.tasks.get(id);
            StringBuilder sb = new StringBuilder();
            for (int i = 0; i < t.es; i++) sb.append(' ');
            for (int i = 0; i < t.duration; i++) sb.append(t.isCritical() ? '#' : '-');
            System.out.printf("%-6s|%s%n", t.id, sb);
        }
    }

    // =====
    // CO2 : String algorithms
    // =====
    static class StringAlgos {

        // ---- KMP: prefix (failure) function + linear-time search, O(n + m) ----
        static int[] prefixFunction(String p) {
            int[] pi = new int[p.length()];
            for (int i = 1, k = 0; i < p.length(); i++) {
                while (k > 0 && p.charAt(i) != p.charAt(k)) k = pi[k - 1];
                if (p.charAt(i) == p.charAt(k)) k++;
                pi[i] = k;
            }
            return pi;
        }

        static List<Integer> kmpSearch(String text, String pat) {
            List<Integer> res = new ArrayList<>();
            if (pat.isEmpty()) return res;
            int[] pi = prefixFunction(pat);
            for (int i = 0, k = 0; i < text.length(); i++) {
                while (k > 0 && text.charAt(i) != pat.charAt(k)) k = pi[k - 1];
                if (text.charAt(i) == pat.charAt(k)) k++;
                if (k == pat.length()) { res.add(i - k + 1); k = pi[k - 1]; }
            }
            return res;
        }

        // ---- Suffix array (sorting suffixes) ----
        static Integer[] suffixArray(String s) {
            int n = s.length();
            Integer[] sa = new Integer[n];
            for (int i = 0; i < n; i++) sa[i] = i;
            Arrays.sort(sa, (a, b) -> {
                int i = a, j = b;
                while (i < n && j < n) {
                    if (s.charAt(i) != s.charAt(j)) return Character.compare(s.charAt(i),
s.charAt(j));
                    i++; j++;
                }
                return Integer.compare(n - a, n - b);
            });
            return sa;
        }

        // ---- LCP array by Kasai's algorithm, O(n): lcp[i] = LCP(sa[i-1], sa[i]) ----
        static int[] lcpArray(String s, Integer[] sa) {
            int n = s.length();
            int[] rank = new int[n], lcp = new int[n];
            for (int i = 0; i < n; i++) rank[sa[i]] = i;
            for (int i = 0, h = 0; i < n; i++) {
                if (rank[i] > 0) {
                    int j = sa[rank[i] - 1];
                    while (i + h < n && j + h < n && s.charAt(i + h) == s.charAt(j + h)) h++;
                    lcp[rank[i]] = h;
                    if (h > 0) h--;
                } else h = 0;
            }
            return lcp;
        }
    }

    static void keywordSearch(DAG g, String keyword) {
        String pat = keyword.toLowerCase();
        int found = 0;
        for (Task t : g.tasks.values()) {
            List<Integer> pos = StringAlgos.kmpSearch(t.name.toLowerCase(), pat);

```

```

        if (!pos.isEmpty()) {
            found++;
            System.out.printf("  %-4s %-22s match at index %s%n", t.id, t.name, pos);
        }
    }
    if (found == 0) System.out.println("  No task name contains \"" + keyword + "\".");
}

// Finds pairs of tasks whose names share a common substring of length >= minLen
static void similarNames(DAG g, int minLen) {
    List<Task> list = new ArrayList<>(g.tasks.values());
    StringBuilder sb = new StringBuilder();
    List<Integer> ownerList = new ArrayList<>();
    for (int t = 0; t < list.size(); t++) {
        String nm = list.get(t).name.toLowerCase();
        for (int i = 0; i < nm.length(); i++) { sb.append(nm.charAt(i)); ownerList.add(t); }
        sb.append((char) (0x100 + t)); // unique separator so matches never cross
    }
    tasks
        ownerList.add(-1);
    String s = sb.toString();
    int[] owner = ownerList.stream().mapToInt(Integer::intValue).toArray();

    Integer[] sa = StringAlgos.suffixArray(s);
    int[] lcp = StringAlgos.lcpArray(s, sa);

    Map<String, int[]> best = new TreeMap<>(); // "a,b" -> {length, start}
    for (int i = 0; i < sa.length; i++) {
        if (owner[sa[i]] < 0) continue;
        int m = Integer.MAX_VALUE;
        for (int j = i + 1; j < sa.length; j++) {
            m = Math.min(m, lcp[j]);
            if (m < minLen) break; // range-minimum of LCP = common prefix
        }
        length
            if (owner[sa[j]] < 0 || owner[sa[j]] == owner[sa[i]]) continue;
            int a = Math.min(owner[sa[i]], owner[sa[j]]), b = Math.max(owner[sa[i]],
owner[sa[j]]);
            String key = a + "," + b;
            if (!best.containsKey(key) || best.get(key)[0] < m) best.put(key, new int[]{m,
sa[i]});
        }
    }
    if (best.isEmpty()) { System.out.println("  No similar task names found (min shared length
" + minLen + ")."); return; }
    for (Map.Entry<String, int[]> e : best.entrySet()) {
        String[] ab = e.getKey().split(",");
        Task x = list.get(Integer.parseInt(ab[0])), y = list.get(Integer.parseInt(ab[1]));
        String shared = s.substring(e.getValue()[1], e.getValue()[1] + e.getValue()[0]);
        System.out.printf("  %s (%s) ~ %s (%s)   shared: \"%s\" (%d chars)%n", x.id, x.name,
y.id, y.name, shared, e.getValue()[0]);
    }
    System.out.println("  -> Please check that these are not duplicate tasks.");
}

// =====
// CO3 / CO1 : Resource planning (single worker, NP-hard in general)
// =====
static final int EXACT_LIMIT = 20;

static void singleWorkerPlan(DAG g, List<String> order) {
    int n = order.size();
    System.out.println("  Problem: ONE worker does all tasks, respecting dependencies.");
    System.out.println("  Goal   : minimise the sum of task completion times (tasks finish as
early as possible).");
    if (n <= EXACT_LIMIT) {
        System.out.println("  Strategy selected: exact BITMASK DP  (n = " + n + " <= " +
EXACT_LIMIT + ", O(2^n * n))");
        bitmaskPlan(g, order);
    } else {
        System.out.println("  Strategy selected: GREEDY heuristic (n = " + n + " > " +
EXACT_LIMIT
            + "; 2^n states is too many, and the problem is NP-hard in general)");
        greedyPlan(g);
    }
}

// dp[mask] = min sum of completion times if exactly the tasks in 'mask' have been done first
static void bitmaskPlan(DAG g, List<String> order) {

```

```

    int n = order.size();
    Map<String, Integer> idx = new HashMap<>();
    for (int i = 0; i < n; i++) idx.put(order.get(i), i);
    int[] dur = new int[n], predMask = new int[n];
    for (int i = 0; i < n; i++) {
        Task t = g.tasks.get(order.get(i));
        dur[i] = t.duration;
        for (String p : t.preds) predMask[i] |= 1 << idx.get(p);
    }
    int full = (1 << n) - 1;
    int[] sum = new int[full + 1];
    long[] dp = new long[full + 1], ways = new long[full + 1];
    int[] parent = new int[full + 1];
    Arrays.fill(dp, Long.MAX_VALUE);
    dp[0] = 0; ways[0] = 1;
    for (int mask = 1; mask <= full; mask++) { int low = Integer.numberOfTrailingZeros(mask);
sum[mask] = sum[mask & (mask - 1)] + dur[low]; }

    for (int mask = 0; mask <= full; mask++) {
        if (dp[mask] == Long.MAX_VALUE) continue;
        for (int i = 0; i < n; i++) {
            if ((mask >> i & 1) != 0 || (predMask[i] & mask) != predMask[i]) continue; // not
ready
            int nm = mask | (1 << i);
            long cost = dp[mask] + sum[nm]; // task i finishes at sum[nm]
            if (cost < dp[nm]) { dp[nm] = cost; parent[nm] = i; }
            ways[nm] += ways[mask]; // counting valid orders
        }
    }
    Deque<String> seq = new ArrayDeque<>();
    for (int m = full; m != 0; m ^= 1 << parent[m]) seq.addFirst(order.get(parent[m]));
    System.out.println(" Best order : " + String.join(" -> ", seq));
    System.out.println(" Sum of finish times : " + dp[full] + " (average " +
String.format("%.2f", dp[full] / (double) n) + " days)");
    System.out.println(" Total time (1 worker) : " + sum[full] + " days");
    System.out.println(" Valid execution orders: " + ways[full] + " (all topological orders,
counted by DP on subsets)");
}

// Heuristic: among ready tasks always do the shortest first
static void greedyPlan(DAG g) {
    Map<String, Integer> deg = new HashMap<>(g.indegree);
    PriorityQueue<Task> ready = new PriorityQueue<>((a, b) -> a.duration != b.duration ?
a.duration - b.duration : a.id.compareTo(b.id));
    for (Task t : g.tasks.values()) if (deg.get(t.id) == 0) ready.add(t);
    List<String> seq = new ArrayList<>();
    long time = 0, sumFinish = 0;
    while (!ready.isEmpty()) {
        Task t = ready.poll();
        seq.add(t.id); time += t.duration; sumFinish += time;
        for (String v : g.adj.get(t.id)) { deg.put(v, deg.get(v) - 1); if (deg.get(v) == 0)
ready.add(g.tasks.get(v)); }
    }
    System.out.println(" Order (heuristic) : " + String.join(" -> ", seq));
    System.out.println(" Sum of finish times : " + sumFinish + " (not guaranteed
optimal)");
    System.out.println(" Total time (1 worker) : " + time + " days");
}

// ---- DP on subsets: split tasks into two teams with balanced workload (subset-sum) ----
static void twoTeamSplit(DAG g) {
    List<Task> list = new ArrayList<>(g.tasks.values());
    int n = list.size(), total = 0;
    for (Task t : list) total += t.duration;
    int half = total / 2;
    boolean[][] can = new boolean[n + 1][half + 1];
    can[0][0] = true;
    for (int i = 1; i <= n; i++) {
        int d = list.get(i - 1).duration;
        for (int s = 0; s <= half; s++) can[i][s] = can[i - 1][s] || (s >= d && can[i - 1][s -
d]);
    }
    int best = half;
    while (!can[n][best]) best--;
    StringBuilder a = new StringBuilder(), b = new StringBuilder();
    int s = best;
    boolean[] inA = new boolean[n];
    for (int i = n; i >= 1; i--) {
        if (can[i - 1][s]) continue;
        inA[i - 1] = true; s -= list.get(i - 1).duration;
    }

```

```

    }
    for (int i = 0; i < n; i++) (inA[i] ? a : b).append(list.get(i).id).append(' ');
    System.out.println(" Team 1: " + a.toString().trim() + " (" + best + " days of work)");
    System.out.println(" Team 2: " + b.toString().trim() + " (" + (total - best) + " days of
work)");
    System.out.println(" Most balanced split: busiest team has " + Math.max(best, total -
best) + " days of work.");
    System.out.println(" (Balances workload only; dependencies may still force waiting between
teams.)");
}

// =====
// CO1 : Problem-class evaluation and strategy selection
// =====
static void strategyReport(int n) {
    System.out.println(" Sub-problem | Problem class
| Strategy chosen");
    System.out.println(" -----+-----
+-----");
    System.out.println(" Valid order of tasks | DAG ordering, polynomial
| Topological sort, O(V+E)");
    System.out.println(" Min. project time, unlimited staff | Longest path in DAG, polynomial
| CPM = DP over topological order");
    System.out.println(" Circular dependency check | Cycle detection in graph
| DFS colouring, O(V+E)");
    System.out.println(" Search tasks by keyword | Substring search
| KMP, O(n+m) - no rescanning");
    System.out.println(" Similar/duplicate task names | Repeated substrings
| Suffix array + LCP array");
    System.out.println(" One worker, min sum of finish time | NP-hard scheduling in general
| " + (n <= EXACT_LIMIT
? "Bitmask DP (n=" + n + " is small enough)" : "Greedy heuristic (n=" + n + " too
large for 2^n)");
    System.out.println(" Split workload between 2 teams | Partition (NP-hard, weakly)
| Subset-sum DP, O(n * total/2)");
    System.out.println(" Assign tasks to skilled workers | Bipartite matching = max-flow |
Edmonds-Karp, O(V*E^2); min-cut check");
    System.out.println(" Finish fast with m workers | NP-hard (contains PARTITION)
| Graham list scheduling, (2-1/m)-approx");
    System.out.println(" Median task duration | Selection
| Las Vegas randomised quick-select");
    System.out.println(" Uncertain durations | Probabilistic estimation
| Monte Carlo simulation");
    System.out.println(" Totals / running sums | Parallel primitives
| Reduce + prefix sum, span O(log n)");
}

// =====
// CO4 : Network flow (Edmonds-Karp) - assign tasks to workers
// =====
static class Worker {
    String name; int cap; Set<String> can = new LinkedHashSet<>();
    Worker(String n, int c) { name = n; cap = c; }
}

static int edmondsKarp(int[][] c, int s, int t) { // BFS augmenting paths, O(V * E^2)
    int n = c.length, total = 0;
    while (true) {
        int[] par = new int[n]; Arrays.fill(par, -1); par[s] = s;
        Deque<Integer> q = new ArrayDeque<>(); q.add(s);
        while (!q.isEmpty() && par[t] < 0) {
            int u = q.poll();
            for (int v = 0; v < n; v++) if (par[v] < 0 && c[u][v] > 0) { par[v] = u; q.add(v); }
        }
        if (par[t] < 0) return total;
        int f = Integer.MAX_VALUE;
        for (int v = t; v != s; v = par[v]) f = Math.min(f, c[par[v]][v]);
        for (int v = t; v != s; v = par[v]) { c[par[v]][v] -= f; c[v][par[v]] += f; }
        total += f;
    }
}

static void assignWorkers(DAG g, List<Worker> ws) {
    List<Task> ts = new ArrayList<>(g.tasks.values());
    int W = ws.size(), T = ts.size(), n = W + T + 2, s = n - 2, t = n - 1;
    int[][] c = new int[n][n];
    for (int w = 0; w < W; w++) {
        c[s][w] = ws.get(w).cap;
    }
}

```

```

        for (int i = 0; i < T; i++) if (ws.get(w).can.contains(ts.get(i).id)) c[w][W + i] = 1;
    }
    for (int i = 0; i < T; i++) c[W + i][t] = 1;
    int[][] orig = new int[n][];
    for (int i = 0; i < n; i++) orig[i] = c[i].clone();
    int flow = edmondsKarp(c, s, t);

    System.out.println(" Network: source -> worker (capacity = max tasks) -> task (1 if
skilled) -> sink");
    System.out.println(" Max flow = " + flow + " of " + T + " tasks assigned");
    for (int w = 0; w < W; w++) {
        List<String> mine = new ArrayList<>();
        for (int i = 0; i < T; i++) if (c[W + i][w] > 0) mine.add(ts.get(i).id);
        System.out.println(" " + ws.get(w).name + " (cap " + ws.get(w).cap + "): " +
(mine.isEmpty() ? "-" : String.join(" ", mine)));
    }
    List<String> left = new ArrayList<>();
    for (int i = 0; i < T; i++) if (c[W + i][t] > 0) left.add(ts.get(i).id);
    System.out.println(" Unassigned tasks: " + (left.isEmpty() ? "none" : String.join(" ",
left)));

    boolean[] inS = new boolean[n]; // min cut = residual-reachable
side
    Deque<Integer> q = new ArrayDeque<>(); q.add(s); inS[s] = true;
    while (!q.isEmpty()) { int u = q.poll(); for (int v = 0; v < n; v++) if (!inS[v] && c[u][v]
> 0) { inS[v] = true; q.add(v); } }
    int cut = 0;
    for (int u = 0; u < n; u++) for (int v = 0; v < n; v++) if (inS[u] && !inS[v]) cut +=
orig[u][v];
    System.out.println(" Min cut capacity = " + cut + " (max-flow = min-cut: " + (cut == flow
? "verified" : "MISMATCH") + ")");
}

static List<Worker> sampleWorkers() {
    List<Worker> ws = new ArrayList<>();
    String[][] d = {"Asha", "2", "A,B,C"}, {"Ravi", "1", "B,D"}, {"Meera", "1", "D,E"};
    for (String[] r : d) { Worker w = new Worker(r[0], Integer.parseInt(r[1]));
w.can.addAll(Arrays.asList(r[2].split(","))); ws.add(w); }
    return ws;
}

static List<Worker> readWorkers(Scanner sc) {
    List<Worker> ws = new ArrayList<>();
    int n = readInt(sc, "Number of workers: ", 1);
    for (int i = 1; i <= n; i++) {
        System.out.print("Worker " + i + " name: "); String nm = sc.nextLine().trim();
        Worker w = new Worker(nm, readInt(sc, " Max tasks: ", 0));
        System.out.print(" Task IDs this worker can do (comma separated): ");
        for (String x : sc.nextLine().trim().toUpperCase().split("\\s*,\\s*")) if
(!x.isEmpty()) w.can.add(x);
        ws.add(w);
    }
    return ws;
}

// =====
// CO5 : NP-hard scheduling -> approximation (Graham list scheduling)
// =====
static void multiWorkerSchedule(DAG g, int m, int cpmTotal) {
    int sum = 0; for (Task t : g.tasks.values()) sum += t.duration;
    System.out.println(" Problem: schedule tasks on " + m + " identical workers, minimise
finish time (makespan).");
    System.out.println(" Class : NP-hard (2 workers, no dependencies = PARTITION), so exact
answers are too costly.");
    System.out.println(" Method : Graham list scheduling (longest ready task first), a
polynomial (2 - 1/" + m + ")-approximation.");
    Map<String, Integer> fin = new HashMap<>();
    int[] free = new int[m];
    List<Task> rem = new ArrayList<>(g.tasks.values());
    int time = 0;
    while (!rem.isEmpty()) {
        List<Task> ready = new ArrayList<>();
        for (Task t : rem) {
            boolean ok = true;
            for (String p : t.preds) { Integer f = fin.get(p); if (f == null || f > time) { ok
= false; break; } }
            if (ok) ready.add(t);
        }
    }
}

```

```

        ready.sort((a, b) -> b.duration != a.duration ? b.duration - a.duration :
a.id.compareTo(b.id));
        for (Task t : ready) {
            int w = -1;
            for (int k = 0; k < m; k++) if (free[k] <= time) { w = k; break; }
            if (w < 0) break;
            fin.put(t.id, time + t.duration); free[w] = time + t.duration; rem.remove(t);
            System.out.printf("    Worker %d: %-3s day %d -> %d\n", w + 1, t.id, time, time +
t.duration);
        }
        int next = Integer.MAX_VALUE;
        for (int f : fin.values()) if (f > time) next = Math.min(next, f);
        if (next != Integer.MAX_VALUE) time = next;
    }
    int makespan = 0; for (int f : fin.values()) makespan = Math.max(makespan, f);
    int lb = Math.max(cpmTotal, (sum + m - 1) / m);
    System.out.println("    Makespan with " + m + " workers : " + makespan + " days");
    System.out.println("    Lower bound on optimum      : " + lb + " days = max(critical path " +
cpmTotal + ", ceil(work " + sum + " / " + m + "))");
    System.out.printf("    Ratio to lower bound      : %.2f (guarantee <= %.2f)\n", makespan /
(double) lb, 2.0 - 1.0 / m);
}

// =====
// CO6 : Randomised + parallel algorithms
// =====
static int quickSelect(int[] a, int k, Random rnd) {          // Las Vegas: always correct,
random running time
    int lo = 0, hi = a.length - 1;
    while (true) {
        if (lo == hi) return a[lo];
        int p = a[lo + rnd.nextInt(hi - lo + 1)], i = lo, j = hi;
        while (i <= j) {
            while (a[i] < p) i++;
            while (a[j] > p) j--;
            if (i <= j) { int tmp = a[i]; a[i] = a[j]; a[j] = tmp; i++; j--; }
        }
        if (k <= j) hi = j; else if (k >= i) lo = i; else return a[k];
    }
}

static void randomisedAndParallel(DAG g, List<String> order) {
    int n = order.size();
    int[] dur = new int[n];
    for (int i = 0; i < n; i++) dur[i] = g.tasks.get(order.get(i)).duration;

    System.out.println("    [Las Vegas] randomised quick-select, expected O(n):");
    System.out.println("    Median task duration = " + quickSelect(dur.clone(), (n - 1) / 2,
new Random(7)) + " days (always correct)");

    System.out.println("    [Monte Carlo] 10000 runs, each duration grows by 0% to 50% at random
(seed 42):");
    Random rnd = new Random(42);
    int trials = 10000, hits = 0, base = 0;
    double[] ef = new double[n]; int[] bestPred = new int[n], crit = new int[n];
    Map<String, Integer> pos = new HashMap<>();
    for (int i = 0; i < n; i++) { pos.put(order.get(i), i); base = Math.max(base,
g.tasks.get(order.get(i)).ef); }
    int deadline = base + 2; double sumEnd = 0;
    for (int r = 0; r < trials; r++) {
        int end = 0;
        for (int i = 0; i < n; i++) {
            Task t = g.tasks.get(order.get(i));
            double es = 0; bestPred[i] = -1;
            for (String p : t.preds) { int pi = pos.get(p); if (ef[pi] > es) { es = ef[pi];
bestPred[i] = pi; } }
            ef[i] = es + t.duration * (1 + 0.5 * rnd.nextDouble());
            if (ef[i] > ef[end]) end = i;
        }
        sumEnd += ef[end];
        if (ef[end] <= deadline) hits++;
        for (int v = end; v >= 0; v = bestPred[v]) crit[v]++;
    }
    System.out.printf("    Expected project time = %.2f days (plain CPM: %d)\n", sumEnd /
trials, base);
    System.out.printf("    P(finish within %d days) = %.1f%%\n", deadline, 100.0 * hits /
trials);
    StringBuilder sb = new StringBuilder("    Criticality index: ");

```

```

        for (int i = 0; i < n; i++) sb.append(order.get(i)).append(' ').append(Math.round(100.0 *
crit[i] / trials)).append("% ");
        System.out.println(sb.toString().trim());

        System.out.println(" [Parallel reduce + prefix sum] on task durations in topological
order:");
        int rounds = 0; for (int s = 1; s < n; s *= 2) rounds++;
        int[] scan = dur.clone();
        for (int d = 1; d < n; d *= 2) { // Hillis-Steele scan: updates
inside a round are independent
            int[] nxt = scan.clone();
            for (int i = d; i < n; i++) nxt[i] = scan[i] + scan[i - d];
            scan = nxt;
        }
        int[] check = dur.clone(); Arrays.parallelPrefix(check, Integer::sum);
        System.out.println(" Prefix sums (one-worker finish times): " + Arrays.toString(scan)
+ (Arrays.equals(scan, check) ? " [matches Arrays.parallelPrefix]" : "
[MISMATCH]"));
        System.out.println(" Total work = " + scan[n - 1] + " days | reduce: " + (n - 1) + "
additions, span " + rounds + " rounds");
        System.out.printf(" Work-span: work = O(n), span = O(log n), parallelism ~ n/log n =
%.1f%n", n / (double) Math.max(1, rounds));
    }

static void runAll(DAG g, List<String> order, int total, List<Worker> ws, Scanner sc) {
    System.out.println("\n[CO2] KMP keyword search for \"database\":"); keywordSearch(g,
"database");
    System.out.println("\n[CO2] Suffix array + LCP - similar task names:"); similarNames(g, 5);
    System.out.println("\n[CO1/CO3] Single worker plan:"); singleWorkerPlan(g, order);
    System.out.println("\n[CO3] Two-team split:"); twoTeamSplit(g);
    System.out.println("\n[CO4] Worker assignment by max-flow:"); assignWorkers(g, ws);
    System.out.println("\n[CO5] Two workers, approximation:"); multiWorkerSchedule(g, 2,
total);
    System.out.println("\n[CO6] Randomised and parallel:"); randomisedAndParallel(g, order);
    System.out.println("\n[CO1] Algorithm selection report:"); strategyReport(order.size());
}

// =====
// Input handling and main
// =====
static DAG readInput(Scanner sc) {
    DAG g = new DAG();
    int n = readInt(sc, "Number of tasks: ", 1);
    for (int i = 1; i <= n; i++) {
        System.out.println("--- Task " + i + " ---");
        String id;
        while (true) {
            System.out.print("ID (e.g. A): ");
            id = sc.nextLine().trim().toUpperCase();
            if (id.isEmpty()) System.out.println("ID cannot be empty.");
            else if (g.tasks.containsKey(id)) System.out.println("Duplicate ID, try again.");
            else break;
        }
        System.out.print("Name: ");
        String name = sc.nextLine().trim();
        int dur = readInt(sc, "Duration (days): ", 0);
        g.addTask(new Task(id, name, dur));
        System.out.print("Depends on (comma separated IDs, or Enter for none): ");
        String line = sc.nextLine().trim().toUpperCase();
        if (!line.isEmpty()) for (String p : line.split("\\s*,\\s*"))
g.tasks.get(id).preds.add(p);
    }
    return g;
}

static int readInt(Scanner sc, String prompt, int min) {
    while (true) {
        System.out.print(prompt);
        try { int v = Integer.parseInt(sc.nextLine().trim()); if (v >= min) return v; }
        catch (NumberFormatException ignored) {}
        System.out.println("Please enter a whole number >= " + min + ".");
    }
}

static DAG sampleProject() {
    DAG g = new DAG();
    String[][] d = {
        {"A", "Requirements", "2", ""},
        {"B", "Database Design", "3", "A"},
    }

```

```

        {"C", "Database Setup", "2", "A"},
        {"D", "Development", "4", "B,C"},
        {"E", "Testing", "2", "D"};
    for (String[] r : d) {
        g.addTask(new Task(r[0], r[1], Integer.parseInt(r[2])));
        if (!r[3].isEmpty()) g.tasks.get(r[0]).preds.addAll(Arrays.asList(r[3].split(",")));
    }
    return g;
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.println("=== PROJECT SCHEDULING SYSTEM (DAG + Topological Sort + CPM) ===");
    System.out.println("1. Enter my own project");
    System.out.println("2. Run the sample project (A-E)");
    int choice = readInt(sc, "Choice: ", 1);
    DAG g = (choice == 2) ? sampleProject() : readInput(sc);
    List<Worker> workers = (choice == 2) ? sampleWorkers() : null;

    for (Task t : g.tasks.values())
        for (String p : t.preds) {
            if (!g.tasks.containsKey(p)) { System.out.println("Error: task " + t.id + " depends
on unknown task '" + p + "'"); return; }
            if (p.equals(t.id)) { System.out.println("Error: task " + t.id + " cannot depend on
itself."); return; }
            g.addEdge(p, t.id);
        }

    List<String> cycle = g.findCycle();
    if (cycle != null) {
        System.out.println("Error: circular dependency detected: " + String.join(" -> ",
cycle));
        System.out.println("A valid schedule is not possible. Please fix the dependencies.");
        return;
    }

    List<String> order = g.topologicalSort();
    int total = runCPM(g, order);
    printSchedule(g, order, total);

    while (true) {
        System.out.println("\n----- MORE TOOLS -----");
        System.out.println("1. Show schedule again");
        System.out.println("2. Search tasks by keyword (KMP)");
        System.out.println("3. Find similar task names (Suffix array + LCP)");
        System.out.println("4. Best order for a single worker (Bitmask DP / greedy)");
        System.out.println("5. Split work between 2 teams (Subset DP)");
        System.out.println("6. Algorithm selection report (problem classes)");
        System.out.println("7. Assign tasks to workers (Max-flow, Edmonds-Karp)");
        System.out.println("8. Schedule on m workers (NP-hard -> approximation)");
        System.out.println("9. Randomised + parallel analysis (Monte Carlo, Las Vegas,
prefix sum)");
        System.out.println("10. Run everything (full demo report)");
        System.out.println("0. Exit");
        int c = readInt(sc, "Choice: ", 0);
        switch (c) {
            case 0: System.out.println("Goodbye!"); return;
            case 1: printSchedule(g, order, total); break;
            case 2: System.out.print("Keyword: "); keywordSearch(g, sc.nextLine().trim());
break;
            case 3: similarNames(g, 5); break;
            case 4: singleWorkerPlan(g, order); break;
            case 5: twoTeamSplit(g); break;
            case 6: strategyReport(order.size()); break;
            case 7: if (workers == null) workers = readWorkers(sc); assignWorkers(g, workers);
break;
            case 8: multiWorkerSchedule(g, readInt(sc, "Number of workers: ", 1), total);
break;
            case 9: randomisedAndParallel(g, order); break;
            case 10: if (workers == null) workers = readWorkers(sc); runAll(g, order, total,
workers, sc); break;
            default: System.out.println("Invalid choice.");
        }
    }
}

```

RESULTS

Screenshots

The program was compiled and run in VS Code (JDK 25) with menu option 2, the sample project A-E.

The screenshot shows an IDE with the following components:

- Explorer:** Shows the project structure with `DSA3PROJECT` containing `ProjectScheduler.java`.
- Editor:** Displays the `ProjectScheduler.java` file. The code defines a `ProjectScheduler` class with a recursive `scan` method that calculates the total work and days for a project, considering dependencies.
- Terminal:** Shows the output of running the program. The output includes:
 - Project Scheduling System (DAG + Topological Sort + CPM) ===
 - 1. Enter my own project
 - 2. Run the sample project (A-E)
 - Choice: 2
 - === EXECUTION ORDER (Topological Sort) ===
 - A -> B -> C -> D -> E
 - === CPM SCHEDULE ===
 - Table with 8 columns: ID, Task, Dur, ES, EF, LS, LF, Slack, Critical.
 - === CRITICAL PATH(S) ===
 - A -> B -> D -> E
 - Minimum project duration: 11 days
 - === TIMELINE (# = critical, - = has slack) ===
 - Timeline output showing critical path A-B-D-E.
 - MORE TOOLS -----
 - 1. Show schedule again

Screenshot 1: Execution order, CPM schedule, critical path, minimum duration and timeline

The screenshot shows a Java IDE with a project named 'DSA3PROJECT'. The main editor displays the 'ProjectScheduler.java' file, which contains a class with a 'schedule' method. The method uses a recursive approach to calculate the minimum project duration and the critical path. The output in the terminal shows the results of the 'schedule' method call.

```
public class ProjectScheduler {
    // ... (method body) ...
    public void schedule() {
        // ... (code) ...
    }
}
```

The terminal output shows the following results:

```
=== CRITICAL PATH(S) ===
A -> B -> D -> E

Minimum project duration: 11 days

=== TIMELINE (# = critical, - = has slack) ===
A | ##
B | ###
C | --
D | ####
E | ##

----- MORE TOOLS -----
1. Show schedule again
2. Search tasks by keyword (KMP)
3. Find similar task names (Suffix array + LCP)
4. Best order for a single worker (Bitmask DP / greedy)
5. Split work between 2 teams (Subset DP)
6. Algorithm selection report (problem classes)
7. Assign tasks to workers (Max-flow, Edmonds-Karp)
8. Schedule on m workers (0-1 hard -> approximation)
9. Randomised + parallel analysis (Monte Carlo, Las Vegas, prefix sum)
10. Run everything (full demo report)
0. Exit
choice: 1
```

Screenshot 2: CPM schedule and the "More tools" menu (options 1-10)

Observations. The execution order is $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E$. Tasks A, B, D and E have slack 0 and form the critical path $A \rightarrow B \rightarrow D \rightarrow E$, so the minimum project duration is **11 days**. Task

C (Database Setup) has 1 day of slack and is shown with "-" in the timeline, while critical tasks are shown with "#".

Output of the analysis modules (menu option 10)

The text below is the console output of "Run everything" on the sample project.

Strings: KMP search and similar names (CO2)

```
[CO2] KMP keyword search for "database":
B   Database Design      match at index [0]
C   Database Setup       match at index [0]

[CO2] Suffix array + LCP - similar task names:
B (Database Design) ~ C (Database Setup)  shared: "database " (9 chars)
-> Please check that these are not duplicate tasks.
```

KMP finds "database" in tasks B and C at index 0. The suffix array with the LCP array finds that B and C share the 9 characters "database ", so the program asks the user to check that they are not duplicates.

Single worker plan and two-team split (CO1 / CO3)

```
[CO1/CO3] Single worker plan:
Problem: ONE worker does all tasks, respecting dependencies.
Goal   : minimise the sum of task completion times (tasks finish as early as possible).
Strategy selected: exact BITMASK DP (n = 5 <= 20, O(2^n * n))
Best order      : A -> C -> B -> D -> E
Sum of finish times : 37 (average 7.40 days)
Total time (1 worker) : 13 days
Valid execution orders: 2 (all topological orders, counted by DP on subsets)

[CO3] Two-team split:
Team 1: A D (6 days of work)
Team 2: B C E (7 days of work)
Most balanced split: busiest team has 7 days of work.
(Balances workload only; dependencies may still force waiting between teams.)
```

With $n = 5$, the exact bitmask DP is selected. The best single-worker order is $A \rightarrow C \rightarrow B \rightarrow D \rightarrow E$ with a sum of finish times of 37, and there are exactly 2 valid orders. The subset-sum DP splits the work into 6 and 7 days.

Worker assignment by max-flow (CO4)

```
[CO4] Worker assignment by max-flow:
Network: source -> worker (capacity = max tasks) -> task (1 if skilled) -> sink
Max flow = 4 of 5 tasks assigned
Asha (cap 2): A, B
Ravi (cap 1): D
Meera (cap 1): E
Unassigned tasks: C
Min cut capacity = 4 (max-flow = min-cut: verified)
```

Max-flow is 4 of 5 tasks. The total worker capacity ($2 + 1 + 1 = 4$) is the bottleneck, so task C is left unassigned. The min-cut capacity is 4, which verifies max-flow = min-cut.

Two workers: approximation (CO5)

```
[CO5] Two workers, approximation:
Problem: schedule tasks on 2 identical workers, minimise finish time (makespan).
Class   : NP-hard (2 workers, no dependencies = PARTITION), so exact answers are too costly.
Method  : Graham list scheduling (longest ready task first), a polynomial  $(2 - 1/2)$ -approximation.
Worker 1: A day 0 -> 2
Worker 1: B day 2 -> 5
Worker 2: C day 2 -> 4
Worker 1: D day 5 -> 9
Worker 1: E day 9 -> 11
Makespan with 2 workers : 11 days
Lower bound on optimum   : 11 days = max(critical path 11, ceil(work 13 / 2))
Ratio to lower bound      : 1.00 (guarantee <= 1.50)
```

Graham list scheduling gives a makespan of 11 days, equal to the lower bound $\max(11, \text{ceil}(13/2)) = 11$, so the ratio is 1.00 (guarantee at most 1.50). A second worker is only useful for C because the critical path dominates.

Randomised and parallel analysis (CO6)

```
[CO6] Randomised and parallel:
[Las Vegas] randomised quick-select, expected O(n):
  Median task duration = 2 days (always correct)
[Monte Carlo] 10000 runs, each duration grows by 0% to 50% at random (seed 42):
  Expected project time = 13.75 days (plain CPM: 11)
  P(finish within 13 days) = 19.4%
Criticality index: A 100% B 100% C 0% D 100% E 100%
[Parallel reduce + prefix sum] on task durations in topological order:
  Prefix sums (one-worker finish times): [2, 5, 7, 11, 13] [matches Arrays.parallelPrefix]
  Total work = 13 days | reduce: 4 additions, span 3 rounds
  Work-span: work = O(n), span = O(log n), parallelism ~ n/log n = 1.7
```

The Las Vegas quick-select gives a median duration of 2 days. In 10000 Monte Carlo runs with random delays of 0% to 50%, the expected duration is 13.75 days against the plain CPM value of 11, and the probability of finishing within 13 days is only 19.4%. The criticality index is 100% for A, B, D, E and 0% for C. The prefix sums [2, 5, 7, 11, 13] match Java's Arrays.parallelPrefix.

Algorithm selection report (CO1)

```
[CO1] Algorithm selection report:
Sub-problem | Problem class | Strategy chosen
-----|-----|-----
Valid order of tasks | DAG ordering, polynomial | Topological sort, O(V+E)
Min. project time, unlimited staff | Longest path in DAG, polynomial | CPM = DP over topological order
Circular dependency check | Cycle detection in graph | DFS colouring, O(V+E)
Search tasks by keyword | Substring search | KMP, O(n+m) - no rescanning
Similar/duplicate task names | Repeated substrings | Suffix array + LCP array
One worker, min sum of finish time | NP-hard scheduling in general | Bitmask DP (n=5 is small enough)
Split workload between 2 teams | Partition (NP-hard, weakly) | Subset-sum DP, O(n * total/2)
Assign tasks to skilled workers | Bipartite matching = max-flow | Edmonds-Karp, O(V*E^2); min-cut
check
Finish fast with m workers | NP-hard (contains PARTITION) | Graham list scheduling, (2-1/m)-
approx
Median task duration | Selection | Las Vegas randomised quick-select
Uncertain durations | Probabilistic estimation | Monte Carlo simulation
Totals / running sums | Parallel primitives | Reduce + prefix sum, span O(log
n)
```

Each sub-problem is classified (polynomial, NP-hard, probabilistic) and the chosen strategy is printed, as shown below.

Summary of results (sample project)

Quantity	Result
Execution order (topological sort)	A → B → C → D → E
Minimum project duration	11 days
Critical path	A → B → D → E (C has 1 day of slack)
One worker: best order / total time	A → C → B → D → E, 13 days (sum of finish times 37, 2 valid orders)
Two-team split	Team 1: A, D (6 days); Team 2: B, C, E (7 days)
Max-flow assignment	4 of 5 tasks assigned (C unassigned); max-flow = min-cut = 4
Two workers (Graham)	11 days = lower bound (ratio 1.00, guarantee 1.50)
Monte Carlo (10000 runs)	Expected 13.75 days; P(finish within 13 days) = 19.4%
Prefix sums of durations	[2, 5, 7, 11, 13]

Conclusion

The Optimized Project Task Scheduling System builds a DAG from the tasks, rejects circular dependencies, orders the tasks with Kahn's algorithm and uses CPM to find the critical path and the minimum duration, which is 11 days for the sample project. Course topics were used where they solve a real scheduling need and not added for their own sake: each sub-problem was first classified as polynomial (topological sort, CPM, max-flow), NP-hard (bitmask DP for small n , approximation for m workers) or probabilistic (Monte Carlo), and the matching algorithm was chosen. The results were checked: max-flow equals min-cut, the prefix sums match Java's `parallelPrefix`, and the small cases were verified by hand.

Future Scope

- A graphical interface with a network diagram of the DAG.
- Reading projects from CSV or Excel files.
- Resource levelling and cost-based crashing of critical tasks.
- PERT with three-point estimates (optimistic, most likely, pessimistic).
- A truly parallel forward pass using Java threads or ForkJoin.
- Rabin-Karp and suffix automaton for large task databases.